

Sysmex CS-2000i: устройство сервиса и работа драйвера

Host Online, .NET 9 / Windows Service. Описание фактического кода, разбор случая R221 от 30.09.2026, исключения, стек вызовов, сборка и размещение файлов.

1. Что изменилось и каковы границы протокола

В активном решении оставлен один протокол - Sysmex Host Online. TCP listener, принятие клиента, состояние сокета и его закрытие принадлежат сервису. Управляемая DLL анализатора работает с уже открытым потоком `IAnalyzerConnection`. Проект ASTM и его тесты удалены из решения. Существующие поля JSON сохранены; основной активный файл - `configs/SysmexCS2000.json`.

Документ `Host Online 2_5197664667366890337.pdf`: стр. 6 требует ASCII; стр. 34-37 задают поля S221, имя пациента длиной 15 символов и код задания из трёх знаков; стр. 38 уточняет, что в задании третья цифра кода группы равна 0. В S221 не предусмотрены дата рождения, пол или произвольная демографическая запись: передать можно только имя пациента в отведённом поле.

Windows Service / `AnalyzerService.Host.exe` → конфигурация, DLL, TCP listener
`AnalyzerService.Contracts` → `IAnalyzerDriver`, `IAnalyzerConnection`, `AnalyzerSettings`
`AnalyzerService.Transport` → общий `TcpHost`, вызываемый только сервисом
`SysmexCS2000.HostOnline.Driver.dll` → R221/S221/D121/D221, кодек, обработка файла
`AnalyzerService.LisDatabase.dll` → SQL-заказы и сопоставление кодов ЛИС
`AnalyzerService.ResultFiles.dll` → сырые `.raw`, отдельный поток, `archive/errors/QC`

2. Запуск: порядок вызовов и владение ресурсами

1. Program создаёт Generic Host с поддержкой Windows Service и регистрирует Worker, `JsonAnalyzerSettingsProvider`, `AnalyzerSettingsValidator`, `AnalyzerManager` и фабрику журналов.
2. Worker.`ExecuteAsync` читает только *.json из верхнего уровня configs, проверяет настройки. При `ActiveStatus=true` вызывает `AnalyzerManager.Add` и затем `StartAllAsync`.
3. `AnalyzerRuntime.StartAsync` создаёт файловый журнал прибора, вызывает `DriverLoader.Load`. `DriverLoadContext` загружает DLL из `DllPath` и зависимости через её `.deps.json`. Созданный `SysmexCS2000HostOnlineDriver` получает `Initialize(logger, settings)`.
4. `IAnalyzerDriver.Start` запускает файловую очередь результатов, если `ResultHandlerStatus=true`. Сервис создаёт `TcpHost` и вызывает `Start(IPaddress, Port)` из JSON. Ни один драйвер не открывает `TcpListener`.
5. `AnalyzerRuntime.RunConnectionsAsync` вызывает `TcpHost.AcceptAsync`. Для принятого клиента создаётся `TcpAnalyzerConnection` с `NetworkStream` и метками RX/TX; затем сервис вызывает `DLL.HandleConnectionAsync(connection, token)`.

Ожидания `AcceptAsync`, `ReadAsync` и `WriteAsync` асинхронны, чтобы не занимать поток службы. Разбор фиксированных полей, SQL-запросы и короткие файловые операции остаются синхронными. Обработка накопленных `.raw` выполняется на отдельном потоке, поэтому не задерживает следующий приём прибора.

3. Стек вызовов на примере запроса со стенда

30.09.2026 15:12:55 RX: R2210101 300926151200000304 9000003160B ...
 Параметры запроса R221: 400, 650, 660, 870, 880; Sample ID: 9000003160
 Журнал БД: найден 1 уникальный код; прежний ответ S221 содержал 392

1. `TcpHost.AcceptAsync` возвращает клиента; `RunConnectionsAsync` передаёт поток в `SysmexCS2000HostOnlineDriver.HandleConnectionAsync`, затем в

AnalyzerSysmexCS2000HostOnline.HandleConnectionAsync.

2. ReadTextAsync читает STX, тело и ETX; AddBlock собирает блоки по номеру. HostOnlineCodec.ParseInquiry извлекает Sample ID, признак регистрации ID и уже показанные прибором коды. Пять кодов в R221 - информация прибора, а не пять найденных назначений из БД.
3. LisDBProvider.GetOrder открывает SQL, получает пациента и рассматривает каждую строку bestall. В протокольном журнале теперь видны исходный код ЛИС, код для прибора, статус, число попыток, наличие результата и причина исключения/включения. Незавершённые коды объединяются по уникальному значению.
4. HostOnlineCodec.BuildOrder превращает исходные коды результата в коды задания. Например, 392 → 390; 051 → 050. Два разных кода группы дают два 9-символьных блока в одном S221. Если два теста отображаются в одну группу, повторный код объединяется - это один групповой заказ для анализатора.
5. BuildHeader переносит ID Information из R221. В вашем сообщении это В (считано по штрихкоду); прежний код всегда писал С (ID назначен хостом). ФИО «ТЕСТ МИХАЛ ИВАН» превращается в ASCII «TEST MIKHAL IVA» после ограничения полем 15 символов. Раньше журнал показывал кириллицу, но Encoding.ASCII на проводе заменял её символами '?'.
6. WriteTextAsync проверяет ASCII и длину кадра ≤255 байт, пишет STX/S221/ETX, отмечает TX и записывает фактически отправляемый текст в протокольный журнал.

По одному фрагменту журнала нельзя доказать, почему SQL вернул ровно один код: возможны уже существующий результат, неподходящий статус/число попыток, отсутствие сопоставления конвапа либо два теста с одним кодом группы. Изменять фильтр на отправку уже выполненных исследований без данных БД нельзя. Новые построчные записи журнала показывают конкретную причину для следующего запроса.

4. Результаты и контроль качества

1. D121/D221 определяется по первому символу сообщения. Код Sample Distinction С на позиции 8 означает контроль качества; байты целого STX...ETX сообщения сохраняются в ResultsFolder/QualityControl без преобразования.
2. Пациентский результат сохраняется как уникальный .raw в ResultsFolder. Сначала пишется .tmp, затем атомарно переименовывается в .raw. Имя включает Sample ID, UTC-время и GUID, чтобы не перезаписывать повторное измерение.
3. Выделенный поток RawResultQueue при старте и далее через сигнал/каждые две секунды читает только готовые *.raw верхнего уровня. ProcessStoredResult восстанавливает тело Host Online, ParseResult извлекает показатели, HostOnlineResultHandler переводит коды через LisDBProvider.TranslateResultCode.
4. При наличии пригодных показателей создаются .res и затем .ok в OutputFolder, после чего исходный .raw переносится в archive. Пустое сообщение, отсутствие сопоставленных тестов, ошибка SQL или записи ведут в errors и файловый Error-журнал. Необработанные файлы переживают перезапуск службы.

ResultHandlerStatus=false оставляет приём и сохранение .raw активными, но не запускает преобразование. При следующем запуске с true накопленные файлы подбираются. Контроли в QualityControl этим потоком не просматриваются: отдельный обработчик контроля качества пока не реализован.

5. Исключения, throw и файловые журналы

Исключение - объект, описывающий ошибку. Конструкция throw new создаёт его и немедленно прерывает обычный путь выполнения. CLR ищет подходящий catch выше по стеку вызовов; при выходе выполняются finally/Dispose. Если catch нет, ошибка поднимается до Generic Host и может остановить службу. В async-методе исключение сохраняется в Task и вновь возникает на await вызывающего метода.

```
if (badCode) throw new InvalidDataException("Недопустимый код");
catch (Exception ex) { logger.Error("Контекст Sample ID", ex); throw; }
throw; сохраняет исходный стек; throw ex; создаёт новую точку стека - избегать.
```

Источник	Как проходит ошибка	Где появляется запись
JSON/DLL/TCP Start	StartAsync логирует Exception и делает throw; Worker получает ошибку	Logs/Error службы и Error анализатора
SQL при R221	GetOrder → HandleConnectionAsync: log + throw → AnalyzerRuntime закрывает клиента	Error анализатора с Sample ID/стеком
Неверный S221	BuildOrder/WriteTextAsync: log + throw; при сетевом сбое серия блоков может оказаться неполной	Error анализатора и состояние TCP
Ошибка .raw-файла	RawResultQueue ловит ошибку отдельного файла, логирует, переносит в errors	Error и Result анализатора; служба продолжает работу
Остановка	Runtime логирует каждую ошибку, затем AggregateException; Worker логирует aggregate и throw	Error анализатора и Logs/Error службы

FileAnalyzerLogger.Error записывает Exception.ToString(): тип, сообщение, стек вызовов и InnerException. Он ведёт отдельные дневные файлы Service, Transport, Protocol, Result и Error в каталоге <служба>/<AnalyzerName>/Logs; ошибки, возникшие до создания логгера прибора, идут в <служба>/Logs/Error. Обычные отмена token и закрытие клиентом потока считаются ожидаемыми событиями, их не следует маскировать под аварии.

6. Остановка: обратный стек вызовов

1. Service Control Manager инициирует Worker.StopAsync; тот вызывает AnalyzerManager.StopAllAsync.
2. AnalyzerRuntime.StopAsync отменяет token и закрывает TcpHost. Текущий AcceptAsync/ReadAsync завершается, RunConnectionsAsync освобождает TcpClient.
3. После завершения сетевой задачи Runtime вызывает IAnalyzerDriver.Stop. RawResultQueue заканчивает текущий файл и соединяет свой выделенный поток через Join.
4. LoadedDriver.Dispose освобождает DLL и контекст загрузки; непереработанные .raw остаются на диске для следующего старта. Ошибки остановки собираются, а не теряются.

7. Сборка в Visual Studio и раскладка файлов

1. Установите Visual Studio 2022 с рабочей нагрузкой .NET desktop и SDK .NET 9. Откройте AnalyzerService.sln. Выберите Release и x64/Any CPU, выполните Restore NuGet и Build Solution. В списке проектов должен быть только Host Online; ASTM отсутствует.
2. Для обычной сборки получатся AnalyzerService.Host.exe и зависимые DLL в bin/Release/net9.0. Для развертывания запустите package.ps1 из PowerShell. Скрипт вызывает dotnet publish с PublishSingleFile=true и PublishTrimmed=false.
3. Однофайловым становится исполняемый хост. Плагин нельзя упаковать внутрь этого EXE без отказа от динамической загрузки: его DLL, .deps.json и зависимости остаются в drivers/SysmexCS2000HostOnline. Публикация framework-dependent: на сервере нужен .NET 9 Runtime. PDB/XML могут лежать рядом как необязательные файлы диагностики.

```
publish/AnalyzerService/AnalyzerService.Host.exe ← один исполняемый файл хоста
publish/AnalyzerService/configs/SysmexCS2000.json ← единственный активный JSON
publish/AnalyzerService/drivers/SysmexCS2000HostOnline/
SysmexCS2000.HostOnline.Driver.dll + .deps.json
AnalyzerService.LisDatabase.dll, AnalyzerService.ResultFiles.dll, SqlClient и другие зависимости
<служба>/<AnalyzerName>/Logs, Results, Results/archive, Results/errors, Results/QualityControl
```

DllPath в JSON задаётся относительно каталога EXE. Под «DLL прибора» в этом решении понимается управляемый плагин SysmexCS2000.HostOnline.Driver.dll. Документы описывают сетевой протокол, но не экспортный API фирменной native DLL; такая DLL в текущем проекте не используется. Для нового прибора создается отдельный плагин IAnalyzerDriver в собственной подпапке drivers с .deps.json и зависимостями.

У учётной записи службы должны быть права чтения configs/drivers, записи в ResultsFolder, OutputFolder и Logs, а также доступ к SQL Server. После изменения DLL перезапустите службу, чтобы DriverLoadContext загрузил новую версию. Относительный OutputFolder разрешается от каталога EXE службы, а не от System32. Две активные конфигурации на одном IP/порту запускать нельзя.

8. Проверка после сборки

```
dotnet build AnalyzerService.sln --maxcpucount:1  
dotnet run --project tests/SysmexCS2000.HostOnline.Driver.Tests --no-build  
./package.ps1 # Release/win-x64, единый EXE + внешняя папка DLL
```

Автономные тесты проверяют конкретный R221, два 9-символьных кода S221, ASCII-транслитерацию и признак ID=B, маршрутизацию QC/пустого результата, файловое логирование пробрасываемого исключения, а также сетевой цикл сервиса с динамически загруженным драйвером. Стендовая проверка с реальной БД всё ещё нужна для определения причины отсутствия второго назначения в запросе 30.09.2026.

ACK/NAK поверх TCP для Host Online в этой доработке не менялись: этот вопрос ранее был отложен до проверки с прибором. Документ-источник протокола: 2_5197664667366890337.pdf, в особенности стр. 6, 29-38 и 40.